

ВСТУП

У сучасних умовах розвитку інформаційних технологій важливим напрямом вдосконалення освітнього процесу є впровадження інформаційних систем, які дозволяють автоматизувати управління навчальною документацією та спростити доступ до необхідної інформації. Одним із важливих етапів підготовки студентів у закладах вищої освіти є виконання курсових, дипломних та кваліфікаційних робіт, що передбачає формування та затвердження відповідних тем.

Традиційно облік і контроль тем студентських робіт здійснюється за допомогою електронних таблиць або паперових документів, що може призводити до дублювання тем, втрати інформації або ускладнювати доступ до актуальних даних для студентів та викладачів. Відсутність централізованої системи управління такими даними знижує ефективність організації цього процесу.

Використання веб-додатків дозволяє створити єдине інформаційне середовище, у якому можна зберігати, переглядати та контролювати інформацію про теми курсових, дипломних та кваліфікаційних робіт. Така система забезпечує швидкий доступ до даних, спрощує процес їх оновлення та дозволяє уникнути помилок, пов'язаних із ручним веденням обліку.

Актуальність теми полягає у необхідності створення зручного інструменту для централізованого моніторингу тем курсових, дипломних та кваліфікаційних робіт, що дозволить підвищити ефективність організації навчального процесу та забезпечити зручний доступ до інформації для студентів і викладачів.

Метою роботи є розробка веб-додатку для моніторингу тем курсових, дипломних та кваліфікаційних робіт, який забезпечить зберігання, перегляд та управління інформацією про теми студентських робіт.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Опис предметної області дослідження

У процесі навчання у закладах вищої освіти одним із важливих етапів підготовки студентів є виконання курсових, дипломних та кваліфікаційних робіт. Ці роботи дозволяють студентам поглибити теоретичні знання, отримані під час навчання, а також набути практичних навичок дослідження, аналізу та вирішення професійних задач. Однією з ключових складових цього процесу є формування, затвердження та облік тем студентських робіт.

Теми курсових, дипломних та кваліфікаційних робіт зазвичай формуються викладачами кафедри або пропонуються студентами з подальшим погодженням із науковим керівником. Після цього теми затверджуються та закріплюються за конкретними студентами. Важливою умовою організації цього процесу є уникнення дублювання тем, а також забезпечення зручного доступу до інформації про вже обрані або доступні теми.

У багатьох випадках облік тем студентських робіт здійснюється за допомогою електронних таблиць, текстових документів або паперових журналів. Такий підхід має ряд недоліків, серед яких складність централізованого зберігання інформації, обмежений доступ до актуальних даних, а також підвищений ризик виникнення помилок під час ручного введення або оновлення інформації. Крім того, за відсутності єдиної інформаційної системи процес пошуку необхідної інформації може займати значний час.

Використання сучасних веб-технологій дозволяє створити спеціалізовані інформаційні системи, які забезпечують централізоване зберігання даних та зручний доступ до них через веб-інтерфейс. Веб-додатки мають ряд переваг, серед яких доступність з різних пристроїв, зручність використання, можливість швидкого оновлення інформації та забезпечення контролю доступу до даних.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Веб-додаток для моніторингу тем курсових, дипломних та кваліфікаційних робіт дозволяє систематизувати інформацію про теми, забезпечити їх зручний перегляд та уникнути дублювання. Така система може містити інформацію про назву теми, студента, наукового керівника, тип роботи та інші необхідні дані.

Таким чином, впровадження веб-додатку для моніторингу тем студентських робіт дозволяє підвищити ефективність організації навчального процесу, спростити управління інформацією та забезпечити зручний доступ до необхідних даних для студентів і викладачів.

1.2 Аналіз зарубіжних та вітчизняних аналогів

У сучасних умовах розвитку інформаційних технологій значна кількість інформаційних систем використовується для організації, зберігання та пошуку даних у різних сферах діяльності. Особливе місце серед них займають системи пошуку та інформаційного моніторингу, які дозволяють швидко знаходити необхідну інформацію серед великої кількості даних. Подібні системи широко застосовуються як у науковій діяльності, так і в освітньому середовищі.

Системи пошуку призначені для обробки, індексації та подальшого знаходження інформації за заданими параметрами або ключовими словами. Основною метою таких систем є забезпечення швидкого доступу користувачів до необхідних даних, а також оптимізація процесу роботи з великими обсягами інформації. Як правило, такі системи використовують бази даних, у яких зберігається структурована інформація, що може бути відсортована, відфільтрована або знайдена за допомогою пошукових запитів.

До зарубіжних аналогів інформаційно-пошукових систем належать різноманітні електронні платформи та веб-сервіси, що забезпечують доступ до структурованої інформації. Такі системи використовують розвинені механізми пошуку, які можуть враховувати ключові слова, категорії, фільтри та інші параметри. Вони забезпечують швидку обробку запитів користувачів і дозволяють

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

ефективно працювати з великими обсягами інформації.

Прикладом однієї з найвідоміших інформаційно-пошукових систем є Google Scholar. Дана система призначена для пошуку наукових публікацій, статей, дисертацій, книг та інших наукових матеріалів. Пошук здійснюється за ключовими словами, іменами авторів, назвами робіт або роком публікації. Система також дозволяє сортувати результати, переглядати цитування та знаходити пов'язані наукові матеріали.

Ще одним прикладом є наукометрична база даних Scopus, яка містить велику кількість наукових публікацій з різних галузей знань. Вона надає можливість виконувати складний пошук інформації за різними параметрами, такими як автор, ключові слова, наукова установа або рік публікації. Система широко використовується у наукових установах та університетах для аналізу наукової діяльності.

Також поширеною системою є Web of Science, яка використовується для пошуку та аналізу наукових публікацій. Вона дозволяє знаходити статті у міжнародних наукових журналах, відстежувати цитування наукових робіт та аналізувати наукову активність дослідників.

Крім спеціалізованих наукових систем, існують і загальні інформаційно-пошукові системи, наприклад Google Search, які використовуються для пошуку інформації у мережі Інтернет. Такі системи індексують величезну кількість веб-сторінок та надають користувачам можливість знаходити інформацію за допомогою ключових слів, фраз або інших параметрів пошуку.

Наведені приклади демонструють ефективність використання інформаційно-пошукових систем для організації доступу до великих масивів даних. Основними їх перевагами є швидкість пошуку, зручність використання, можливість фільтрації інформації та структуроване представлення результатів. Подібні принципи можуть бути використані і під час розробки веб-додатку для моніторингу тем курсових, дипломних та кваліфікаційних робіт, що дозволить забезпечити швидкий та зручний пошук необхідної інформації.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Вітчизняні аналоги подібних інформаційних систем також активно розвиваються та використовуються у різних сферах діяльності, зокрема в освіті, державному управлінні та наукових дослідженнях. В Україні такі системи впроваджуються для автоматизації процесів обробки інформації, спрощення доступу до даних та підвищення ефективності роботи установ.

У сфері освіти інформаційні системи часто використовуються для управління навчальним процесом, ведення електронних журналів, формування розкладу занять, обліку успішності студентів та зберігання різноманітної навчальної документації. Одним із прикладів подібних систем є Єдина державна електронна база з питань освіти, яка використовується для зберігання та обробки інформації про студентів, навчальні заклади та освітні програми. Дана система забезпечує централізований доступ до освітніх даних та використовується для адміністрування багатьох процесів у сфері освіти.

Також у навчальних закладах можуть використовуватися різноманітні електронні платформи для організації навчального процесу, наприклад Moodle. Ця система дозволяє викладачам розміщувати навчальні матеріали, проводити тестування студентів, організовувати комунікацію та контролювати виконання навчальних завдань. Подібні платформи значно спрощують управління освітнім процесом та забезпечують доступ до навчальних ресурсів через мережу Інтернет.

Проте більшість таких систем орієнтовані на загальне управління навчальним процесом і не мають спеціалізованого функціоналу для обліку та моніторингу тем курсових, дипломних та кваліфікаційних робіт. Інформація про теми студентських робіт часто зберігається у вигляді окремих документів або електронних таблиць, що ускладнює процес пошуку, оновлення та контролю даних.

Крім того, у таких підходах можуть виникати проблеми з дублюванням тем, відсутністю централізованого доступу до актуальної інформації та складністю швидкого пошуку необхідних даних. У зв'язку з цим виникає потреба у створенні спеціалізованого веб-додатку, який дозволить ефективно здійснювати моніторинг тем студентських робіт, забезпечувати їх структуроване зберігання та зручний

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

пошук інформації для студентів і викладачів.

Сучасні інформаційно-пошукові системи характеризуються високою швидкістю обробки даних, можливістю централізованого зберігання інформації та зручним інтерфейсом для користувача. Використання веб-технологій забезпечує доступ до таких систем з різних пристроїв, включаючи комп'ютери, ноутбуки, планшети та смартфони, що значно підвищує мобільність та доступність інформації. Крім того, веб-системи дозволяють оперативно оновлювати дані та гарантують постійний доступ до актуальної інформації.

Важливим аспектом є організація даних у структурованих базах, що забезпечує ефективний пошук та сортування інформації за різними критеріями. Користувачі можуть шукати дані за ключовими словами, категоріями або іншими параметрами, а також переглядати результати у відсортованому або згрупованому вигляді, що значно економить час та спрощує роботу з інформацією.

Попри широку доступність таких систем, більшість із них орієнтовані на загальні задачі управління даними або підтримки навчального процесу. Зазвичай вони дозволяють зберігати навчальні матеріали, керувати курсами, вести облік студентів та контролювати успішність, проте не пропонують спеціалізованих інструментів для обліку та контролю тем курсових, дипломних чи кваліфікаційних робіт.

У багатьох навчальних закладах інформація про теми студентських робіт ведеться у вигляді окремих документів або електронних таблиць, що ускладнює доступ до даних і підвищує ризик помилок та дублювання. Відсутність єдиної централізованої системи ускладнює пошук актуальних тем і створює додаткові труднощі для студентів та викладачів при організації та контролі виконання робіт.

Тому актуальною є розробка спеціалізованого веб-додатку, який забезпечить централізоване зберігання інформації про теми студентських робіт, зручний інтерфейс для пошуку та перегляду даних, а також полегшить управління інформацією у межах навчального закладу, підвищуючи ефективність організації навчального процесу.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

1.3 Обґрунтування необхідності розробки

Сучасний освітній процес у закладах вищої освіти передбачає виконання студентами курсових, дипломних та кваліфікаційних робіт, які є важливою складовою підготовки фахівців та набуття практичних навичок дослідження і аналізу професійних задач. Одним із ключових аспектів організації цього процесу є своєчасний облік та контроль тем робіт, що дозволяє уникнути дублювання, забезпечити рівномірне навантаження на викладачів та підтримувати актуальність даних про студентські дослідження.

Наразі у більшості навчальних закладів процес обліку тем здійснюється за допомогою паперових журналів, електронних таблиць або окремих документів. Такий підхід не забезпечує централізованого зберігання інформації, ускладнює швидкий пошук та аналіз даних, а також підвищує ризик втрати інформації через людський фактор. Додатково це створює труднощі при контролі прогресу виконання робіт, оскільки викладачі витрачають значну кількість часу на узгодження тем, перевірку їх унікальності та оновлення даних вручну.

З огляду на ці проблеми виникає потреба у створенні спеціалізованого веб-додатку, який би дозволяв централізовано зберігати інформацію про теми студентських робіт та забезпечував зручний доступ до неї для всіх учасників навчального процесу. Така система повинна включати не лише перелік тем, але й дані про студентів, наукових керівників, статус затвердження робіт, а також інші параметри, що дозволяють вести повноцінний моніторинг та контроль.

Переваги розробки веб-додатку для моніторингу тем студентських робіт включають:

- централізацію даних, що дозволяє зберігати всю інформацію в єдиній системі, зменшуючи ризик втрати або дублювання тем;

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

- покращену доступність інформації, оскільки користувачі можуть працювати з додатком через веб-інтерфейс на різних пристроях у будь-який час;
- зручність управління та контролю, що дозволяє викладачам оперативно відстежувати процес затвердження тем, зміни статусу робіт та прогрес студентів;
- економію часу, адже пошук необхідної інформації відбувається за кілька секунд за допомогою фільтрів, сортування або ключових слів;

Підвищення якості організації навчального процесу, завдяки структурованому обліку тем і мінімізації помилок, пов'язаних з ручним веденням даних.

Крім того, впровадження подібної системи створює можливість інтеграції з іншими електронними платформами навчального закладу, такими як системи електронного журналу або платформи дистанційного навчання, що додатково підвищує ефективність інформаційного обміну та управління освітнім процесом.

Таким чином, розробка спеціалізованого веб-додатку є актуальною та необхідною, оскільки він не лише спрощує облік та контроль тем студентських робіт, а й забезпечує сучасний підхід до організації навчального процесу, підвищує ефективність роботи викладачів та полегшує доступ студентів до необхідної інформації.

1.4 Постановка задачі на проєктування

Сучасні заклади вищої освіти стикаються з низкою складнощів у процесі управління дипломними проєктами студентів. Традиційні методи обліку та контролю, які передбачають використання Excel-таблиць, текстових документів або паперових журналів, мають суттєві обмеження. Інформація про теми, студентів та керівників розпорошена по різних носіях, що ускладнює швидкий доступ до актуальних даних та робить процес отримання повної картини виконання

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

дипломних проєктів надзвичайно трудомістким.

Крім того, ручне ведення обліку ускладнює пошук і фільтрацію інформації. Виявлення конкретного дипломного проєкту за заданими критеріями, наприклад, за керівником, групою або статусом виконання, вимагає перебору великої кількості рядків або документів, що значно уповільнює робочий процес та збільшує ймовірність помилок. У випадку відсутності централізованої системи виникає підвищений ризик дублювання тем, втрати важливої інформації або неправильного призначення керівників. Також без інтегрованого інструменту важко контролювати прогрес виконання проєктів, призначати дедлайни або відслідковувати стан кожного етапу, що негативно впливає на планування та організацію навчального процесу.

Ще однією проблемою є обмежені можливості аналітики. На основі розрізнених і несистематизованих даних складно формувати звіти про кількість студентів, теми проєктів, завантаженість викладачів та ефективність кафедри. В результаті це створює додаткові труднощі для адміністрації та викладачів, уповільнює прийняття рішень і підвищує ймовірність організаційних помилок.

У зв'язку з наведеними проблемами виникає необхідність створення спеціалізованого веб-застосунку для централізованого моніторингу та управління дипломними проєктами. Основна мета розробки полягає в тому, щоб забезпечити єдиний інструмент для відстеження стану всіх проєктів, що дозволить адміністрації та викладачам швидко отримувати актуальну інформацію, контролювати прогрес виконання робіт і ефективно організовувати навчальний процес.

Веб-застосунок має стати інтегрованим середовищем для роботи з даними, у якому зберігатимуться відомості про студентів, наукових керівників та теми дипломних проєктів. Така централізація забезпечить підвищення точності інформації та спрощення процесу її оновлення. Крім того, система повинна дозволяти здійснювати пошук і фільтрацію проєктів за різними параметрами, такими як група, курс, семестр, керівник або статус виконання, що значно підвищить ефективність обробки інформації та зекономить час користувачів.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Важливим аспектом є можливість контролювати стан проєктів на різних етапах – від планування до захисту, включаючи встановлення та редагування дедлайнів. Також система має підтримувати управління взаємозв'язками між студентами, темами та керівниками, що дозволить уникнути дублювання тем і конфліктів при закріпленні проєктів. Для зручності перенесення існуючих даних необхідна функція імпорту інформації з Excel-файлів, що дозволить швидко інтегрувати наявні дані в систему та забезпечити їх регулярне оновлення.

Крім цього, веб-застосунок повинен надавати аналітичні можливості, формуючи звіти про кількість дипломних проєктів за групами, статистику завантаженості керівників та стан виконання робіт. Це допоможе адміністрації та викладачам приймати обґрунтовані рішення, планувати завантаження кафедри та контролювати прогрес студентів.

Не менш важливою є мобільність і доступність системи. Веб-орієнтоване рішення забезпечує роботу з будь-якого пристрою, включаючи комп'ютери, ноутбуки, планшети та смартфони, що дозволяє користувачам отримувати доступ до інформації у будь-який час і з будь-якого місця.

Для досягнення поставленої мети веб-застосунок повинен забезпечити комплексне управління інформацією про дипломні проєкти. Він має реалізувати облік студентів та керівників, зберігаючи можливість створення, редагування та видалення записів. Також система повинна дозволяти управляти темами проєктів, включаючи додавання нових тем, редагування описів, зміни статусу та прив'язку до конкретного студента і керівника.

Інтеграція з Excel-файлами забезпечить швидке перенесення та оновлення даних, що значно зменшить трудовитрати на ручний введення інформації. Розширений пошук та фільтрація дозволять швидко знаходити дипломні проєкти за різними критеріями, а візуалізація прогресу виконання робіт у зручному форматі (таблиці, графіки) надасть користувачам можливість оцінювати стан проєктів на кожному етапі.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Додатково система повинна підтримувати контроль дедлайнів для кожного етапу роботи, забезпечувати нагадування про наближення термінів і формувати аналітичні звіти для керівництва. Такий підхід дозволить ефективно планувати роботу кафедри, відслідковувати завантаженість викладачів та прогрес студентів, а також підвищить загальну ефективність організації навчального процесу.

Впровадження спеціалізованого веб-застосунку для моніторингу та управління дипломними проєктами у закладах вищої освіти забезпечує суттєве покращення організації навчального процесу та підвищення ефективності роботи кафедр. По-перше, централізоване зберігання даних про студентів, наукових керівників та теми дипломних проєктів дозволяє отримувати актуальну інформацію у будь-який час і з будь-якого пристрою. Це істотно зменшує ризик помилок, дублювання записів і втрати інформації, що є характерним для традиційних методів обліку, таких як Excel-таблиці чи паперові журнали.

По-друге, веб-застосунок значно підвищує зручність управління та контролю. Викладачі отримують можливість швидко відстежувати процес затвердження тем, зміну статусу робіт та прогрес студентів на кожному етапі виконання проєкту. Можливість встановлення та контролю дедлайнів дозволяє ефективно планувати завантаження кафедри, а студентам — орієнтуватися у термінах та вимогах. Розширені функції пошуку, фільтрації та сортування інформації економлять час користувачів і дозволяють знаходити необхідні дані буквально за кілька секунд, що підвищує оперативність прийняття рішень.

По-третє, система надає потужні аналітичні можливості. Генерація статистичних звітів, графіків завантаженості викладачів, кількості проєктів за групами або курсами дозволяє керівництву кафедри оцінювати ефективність розподілу ресурсів та своєчасно виявляти вузькі місця у навчальному процесі. Це не лише підвищує прозорість процесів, а й сприяє оптимізації управлінських рішень та планування роботи кафедри.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Важливою перевагою веб-застосунку є його інтеграційний потенціал. Система може взаємодіяти з іншими електронними платформами навчального закладу, такими як електронний журнал, системи дистанційного навчання або освітні ресурси, що створює єдине інформаційне середовище. Така інтеграція сприяє ефективнішому обміну даними, автоматизації рутинних адміністративних завдань і полегшує координацію між студентами та викладачами.

Крім того, використання веб-орієнтованого рішення забезпечує мобільність і гнучкість роботи користувачів. Студенти та викладачі можуть отримувати доступ до даних на комп'ютерах, ноутбуках, планшетах або смартфонах у будь-який зручний час, що особливо актуально в умовах дистанційного або змішаного навчання. Це підвищує доступність інформації та створює умови для більш ефективної комунікації та організації навчального процесу.

Нарешті, впровадження такого веб-застосунку формує основу для подальшого розвитку цифрової інфраструктури університету. Система забезпечує сучасний підхід до обліку та контролю дипломних проєктів, підвищує якість організації навчального процесу, сприяє більш прозорій роботі кафедр та дозволяє розширювати функціонал у майбутньому, наприклад, за рахунок інтеграції з іншими освітніми або аналітичними системами.

Отже, розробка та впровадження веб-застосунку для управління дипломними проєктами є не лише актуальною, а й стратегічно важливою для цифровізації освітнього процесу, оптимізації роботи викладачів та підвищення ефективності навчання студентів у закладі вищої освіти.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

1.5 Додаткові аспекти організації та перспективи впровадження веб-застосунку

Впровадження спеціалізованого веб-застосунку для управління дипломними проектами має значний вплив на організацію навчального процесу у закладах вищої освіти. Централізоване управління даними про студентів, керівників та теми дипломних робіт дозволяє ефективніше планувати роботу кафедр, розподіляти навантаження викладачів і забезпечувати своєчасний контроль виконання проєктів. Завдяки інтегрованій системі значно знижується адміністративне навантаження на викладачів: вони витрачають менше часу на пошук та перевірку інформації, а процес отримання даних стає прозорим та оперативним. Студенти отримують можливість відстежувати стан своїх робіт на кожному етапі, бачити дедлайни та коментарі наукових керівників, що підвищує їхню залученість, дисциплінованість та відповідальність за власні результати.

Впровадження веб-застосунку вирішує низку практичних проблем. Насамперед зменшується ризик дублювання тем та конфліктів між студентами і керівниками, що раніше виникав через недостатню централізацію даних. Система забезпечує контроль якості та актуальності інформації, яку раніше вводили вручну, а також дає змогу оперативно реагувати на зміни: коригувати дедлайни, замінювати керівників або додавати необхідні ресурси. Наприклад, у випадку відсутності одного з керівників або зміни навчальної групи, система автоматично оновлює відповідні записи та повідомляє зацікавлених користувачів, що запобігає затримкам у виконанні робіт. Це дозволяє підтримувати високу точність обліку та ефективність управління процесом виконання дипломних проєктів.

Соціально-технічні аспекти впровадження веб-застосунку включають інтеграцію нової системи у щоденну роботу кафедри та організацію навчання персоналу і студентів щодо її використання. Викладачі отримують чіткі інструкції щодо внесення та оновлення даних, а студенти вчаться орієнтуватися у цифровому

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

середовищі, що формує навички самостійного контролю та планування.

Важливим фактором є також мотивація користувачів до роботи з платформою, що досягається за рахунок інтуїтивного та зручного інтерфейсу, швидкого доступу до даних і простоти використання. Система дозволяє користувачам налаштовувати повідомлення про наближення дедлайнів, отримувати нагадування про завдання і бачити прогрес роботи у візуальному вигляді (наприклад, через графіки та таблиці), що робить процес навчання більш прозорим та контрольованим.

Перспективи розвитку системи значні. Можливе подальше розширення функціоналу для обліку курсових, наукових та грантових робіт, а також інтеграція з платформами дистанційного навчання та електронними журналами. Це дозволить створити єдине інформаційне середовище для студентів та викладачів, де всі дані про навчальні роботи зберігаються централізовано і доступні у режимі реального часу. Аналітичні можливості системи можуть бути розширені до рівня факультету або університету, що дозволить здійснювати стратегічне планування, оцінювати ефективність розподілу ресурсів і своєчасно виявляти вузькі місця у навчальному процесі. Наприклад, керівництво кафедри зможе аналізувати завантаженість викладачів, виявляти теми, що повторюються, або визначати групи студентів, які потребують додаткового контролю та підтримки.

Таким чином, створення та впровадження веб-застосунку не лише вирішує нагальні проблеми обліку та контролю дипломних проєктів, але й формує основу для подальшої цифровізації навчального процесу. Це стратегічно важливий крок, що підвищує ефективність роботи кафедр, оптимізує навантаження на викладачів, забезпечує студентам доступ до актуальної та якісної інформації у будь-який зручний час, а також відкриває перспективи для розвитку нових освітніх та аналітичних сервісів на базі університетської інформаційної інфраструктури.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

2 ПРОЄКТУВАННЯ ТА ВИБІР ТЕХНОЛОГІЙ

2.1 Аналіз основних характеристик веб сервісу та процесу пошуку

Веб-сервіс є ключовим елементом сучасних інформаційних систем, що забезпечує взаємодію між клієнтами та сервером через мережу. Його основна функція полягає у наданні клієнтським застосункам доступу до даних та логіки обробки інформації у централізованій формі. Це дозволяє реалізувати складні сценарії взаємодії між користувачем і системою без потреби встановлювати локальні копії бази даних або логіки обробки на стороні клієнта. Для передачі інформації між клієнтом і сервером використовуються стандартні протоколи HTTP та HTTPS. Протокол HTTP забезпечує надійну передачу запитів та отримання відповідей від сервера, тоді як HTTPS додає рівень безпеки завдяки шифруванню даних, що критично важливо при роботі з персональною інформацією студентів та викладачів. Формати обміну даними, такі як JSON (JavaScript Object Notation) та XML (Extensible Markup Language), дозволяють представляти складну інформацію у структурованому та зручному для обробки вигляді. JSON є більш сучасним і легким у використанні для веб-додатків завдяки своїй сумісності з JavaScript та компактному представленню даних, тоді як XML використовується у випадках, коли потрібна сувора структуризація та перевірка даних за схемою.

Архітектурно сучасні веб-сервіси часто реалізуються за принципами REST (Representational State Transfer). Використання REST забезпечує кілька ключових переваг. По-перше, створюється уніфікований інтерфейс доступу до ресурсів системи через стандартні HTTP-методи (GET для отримання даних, POST для створення нових записів, PUT для оновлення та DELETE для видалення). По-друге, REST дозволяє реалізувати stateless-взаємодію: кожен запит до сервера містить всю необхідну інформацію для його обробки, незалежно від попередніх запитів. Це спрощує масштабування системи та дозволяє обробляти велику кількість

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

одночасних запитів без необхідності зберігати стан клієнта на сервері.

Важливими характеристиками веб-сервісу є також клієнт-серверна архітектура та незалежність від платформи. Клієнтська частина, зазвичай реалізована як веб-інтерфейс у браузері, відповідає за взаємодію з користувачем, відображення інформації та відправку запитів до серверу. Серверна частина забезпечує обробку запитів, доступ до бази даних, логіку перевірки та зберігання інформації. Такий розподіл функцій дозволяє незалежно модернізувати або замінювати клієнтську або серверну частину без порушення роботи системи.

Процес пошуку у системі є однією з ключових функцій, що безпосередньо впливає на ефективність роботи користувачів. Пошук реалізується через спеціальні API-ендпоінти, які отримують запит користувача у вигляді HTTP GET-запиту. Користувач може задавати ключові слова для пошуку, а також застосовувати фільтри за такими параметрами, як навчальна група, курс, семестр, науковий керівник або статус дипломного проєкту. Система обробляє запит, здійснює пошук у базі даних і повертає релевантні результати у структурованому форматі.

Для підвищення точності та корисності результатів застосовується багатокритеріальний підхід. Спочатку здійснюється пошук точних збігів між введеним запитом та відповідними полями у записах бази даних (назва теми, ПІБ студента, ПІБ керівника). Якщо точного збігу не виявлено, система переходить до нечіткого пошуку, що дозволяє знаходити записи навіть при частковому збігу слів або неточному введенні даних. Наприклад, якщо студент або викладач неправильно ввів частину імені, система все одно здатна запропонувати найбільш релевантні результати, пріоритетизуючи записи за кількістю збігів та їх позицією в тексті.

Такий підхід забезпечує високу ефективність пошуку та мінімізує ризик пропуску потрібних даних. Загалом веб-сервіс у контексті даного проєкту виконує роль єдиної точки доступу до централізованих даних про дипломні проєкти, забезпечуючи швидкий, зручний і надійний пошук, фільтрацію та обробку інформації.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Його використання дозволяє підвищити прозорість навчального процесу, оптимізувати час користувачів та зменшити ймовірність помилок, характерних для ручного введення даних.

2.2 Визначення функціональних та нефункціональних вимог

У сучасних інформаційних системах визначення вимог є ключовим етапом проектування, який безпосередньо впливає на ефективність, стабільність і корисність кінцевого продукту. В контексті розробки веб-застосунку для моніторингу дипломних проєктів вимоги визначають очікувану поведінку системи та характеристики її роботи, забезпечуючи відповідність функціоналу потребам користувачів та стандартам сучасних веб-технологій.

Функціональні та нефункціональні вимоги виконують різні, але взаємопов'язані ролі у процесі розробки. Функціональні вимоги визначають конкретні дії та операції, які система повинна виконувати. Вони відповідають на питання «Що система повинна робити?» і описують можливості користувачів щодо взаємодії із застосунком. Функціональні вимоги допомагають точно сформулювати завдання для розробників та тестувальників, визначити сценарії використання та логіку бізнес-процесів.

Нефункціональні вимоги, у свою чергу, визначають якісні характеристики та обмеження системи, що не пов'язані безпосередньо з конкретними операціями, але впливають на її ефективність, безпеку та зручність. Вони відповідають на запитання «Як система повинна працювати?» і охоплюють такі аспекти, як продуктивність, масштабованість, безпека, доступність, надійність та юзабіліті. Нefункціональні вимоги забезпечують, щоб система була не лише працездатною, а й ефективною та стабільною під час експлуатації.

У контексті веб-застосунку для моніторингу дипломних проєктів обидві категорії вимог взаємопов'язані. Функціональний аспект гарантує, що користувачі

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

зможуть додавати, редагувати, шукати та відслідковувати дипломні роботи, тоді як нефункціональні вимоги забезпечують швидку обробку запитів, безпечний доступ до даних, високу стабільність системи та зручність інтерфейсу для студентів і викладачів. Ігнорування будь-якої з цих категорій може призвести до створення системи, яка формально виконує операції, але при цьому є незручною, повільною або ненадійною, що істотно знижує її практичну цінність для користувачів.

Крім того, чітке визначення вимог на ранньому етапі розробки дозволяє:

- Упорядкувати процес проєктування та зменшити ймовірність помилок під час кодування та тестування;
- забезпечити узгодженість очікувань користувачів та функціоналу системи, що особливо важливо у навчальному середовищі, де різні групи користувачів мають різні потреби;
- спланувати подальший розвиток системи, інтеграцію нових функцій та модернізацію існуючих модулів без порушення архітектури застосунку;
- забезпечити контроль якості та аудит, оскільки чітко сформульовані вимоги слугують критерієм для тестування та приймання системи.

Таким чином, розмежування функціональних і нефункціональних вимог дозволяє створити системний підхід до розробки веб-застосунку, який буде не лише корисним та зручним для користувачів, а й технічно надійним, масштабованим і безпечним. Це забезпечує основу для детальної розробки архітектури системи, планування бази даних, вибору технологій та інструментів реалізації, а також створення зрозумілих сценаріїв використання для студентів, викладачів та адміністративного персоналу навчального закладу.

2.2.1 Функціональні вимоги

Функціональні вимоги веб-застосунку визначають не лише дії, які система здатна виконувати, а й способи взаємодії між користувачами та даними, що дозволяють підвищити ефективність навчального процесу.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Вони формують логіку бізнес-процесів і створюють передумови для прозорого контролю роботи студентів та керівників.

Організація та контроль інформації: веб-застосунок повинен централізувати всі дані про дипломні проекти, уникаючи дублювання та конфліктів. Система реалізує механізми, які забезпечують зв'язок між студентами, темами проектів і керівниками, а також контроль за станом виконання роботи. Кожен запис у базі даних має чітко визначені атрибути та зв'язки, що дозволяє відстежувати прогрес без необхідності ручного узгодження даних.

Моніторинг та сповіщення: функціонал системи передбачає автоматизоване сповіщення про критичні події: наближення дедлайнів, зміни керівника, додаткові ресурси або зміни статусу проекту. Це не лише оптимізує час викладачів та студентів, а й зменшує ймовірність пропуску важливих етапів, що часто трапляється при ручному контролі.

Пошук та аналітика: система забезпечує багаторівневий пошук і фільтрацію даних за різними критеріями: група, курс, науковий керівник, семестр або статус роботи. Використання багатокритеріального та нечіткого пошуку дозволяє отримувати релевантні результати навіть при неповній або неточній інформації. Додатково веб-застосунок формує аналітичні звіти: статистику завантаженості викладачів, прогрес студентів та узагальнені показники виконання робіт по кафедрах або факультету.

Інтеграція та масштабованість: функції імпорту даних із зовнішніх джерел (наприклад, Excel) та інтеграції з системами дистанційного навчання забезпечують швидкий запуск та оновлення інформації без дублювання. Система спроектована так, щоб у майбутньому можна було додавати нові модулі: для курсових робіт, грантових проектів або розширеної аналітики.

Управління доступом та безпека: кожен користувач отримує роль з визначеними правами доступу, що запобігає несанкціонованому редагуванню даних. Система також зберігає історію змін, що дозволяє відстежувати, хто і коли вносив корективи, підвищуючи прозорість процесу і надійність роботи.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

2.2.2 Нефункціональні вимоги

Нефункціональні вимоги характеризують загальні умови та параметри, у межах яких повинна працювати інформаційна система. Вони описують особливості організації її роботи, технічні обмеження та вимоги до якості функціонування. Такі вимоги визначають характеристики середовища виконання, рівень стабільності роботи, особливості взаємодії з користувачами та загальні принципи забезпечення коректної роботи застосунку.

Продуктивність та швидкодія: система повинна обробляти запити користувачів із мінімальною затримкою, навіть при одночасній роботі великої кількості студентів та викладачів. Оптимізовані алгоритми пошуку та кешування результатів дозволяють забезпечити швидке відображення інформації про дипломні проекти, що підвищує ефективність користування та економить час на пошук даних.

Надійність та стійкість до помилок: веб-застосунок проєктується таким чином, щоб критичні функції (облік тем, контроль дедлайнів, сповіщення) залишалися працездатними навіть у разі часткових збоїв серверу або проблем із мережею. Система повинна автоматично зберігати проміжні зміни, вести журнал помилок та відновлювати роботу після непередбачуваних ситуацій без втрати даних.

Безпека та захист даних: нефункціональні вимоги передбачають надійний контроль доступу до конфіденційної інформації: персональні дані студентів, оцінки, коментарі керівників. Реалізуються механізми шифрування з'єднань (HTTPS), аутентифікації та авторизації за ролями, обмеження прав редагування та історія змін для аудиту. Це гарантує, що дані не будуть втраченою або використані неправомірно.

Юзабіліті та зручність інтерфейсу: інтерфейс системи повинен бути інтуїтивно зрозумілим для студентів і викладачів різного рівня технічної

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

підготовки. Продумана навігація, підказки, доступ до основних функцій у кілька кліків та адаптивний дизайн під різні пристрої дозволяють знизити час навчання користувачів і підвищити їхню мотивацію до активного використання системи.

Масштабованість та можливість розширення: архітектура застосунку повинна дозволити додавати нові модулі та інтегруватися з іншими системами навчального закладу без порушення існуючої роботи. Це забезпечує довгострокову перспективу розвитку: підключення нових баз даних, модулів для курсових та наукових проєктів, розширення аналітики для факультету та університету.

Доступність та підтримка високого навантаження: система повинна залишатися доступною цілодобово із забезпеченням стабільного рівня обслуговування при пікових навантаженнях. Використання сучасних серверних технологій та балансування запитів дозволяє запобігати затримкам або відмовам у роботі системи під час активного використання.

2.3 Архітектура веб-сервісу та взаємодія компонентів

Архітектура веб-сервісу є однією з ключових складових під час розробки сучасних інформаційних систем, оскільки саме вона визначає спосіб організації програмних компонентів, принципи їхньої взаємодії, розподіл відповідальності між окремими модулями та загальну логіку функціонування застосунку. Для системи моніторингу дипломних проєктів правильне проєктування архітектури має особливе значення, оскільки така система повинна забезпечувати централізоване зберігання даних, швидкий пошук інформації, обробку запитів від різних категорій користувачів, а також стабільну і безпечну роботу в умовах постійного оновлення інформації.

У загальному випадку архітектура веб-сервісу будується за принципом клієнт-серверної взаємодії. Такий підхід передбачає поділ системи на дві основні частини: клієнтську та серверну. Клієнтська частина відповідає за взаємодію з користувачем, відображення даних, обробку дій у графічному інтерфейсі та

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

формування запитів до серверної частини. Серверна частина, у свою чергу, виконує обробку отриманих запитів, реалізує бізнес-логіку, здійснює перевірку даних, взаємодіє з базою даних та повертає результати клієнту у визначеному форматі.

Для розроблюваного веб-застосунку така архітектура є доцільною, оскільки дозволяє чітко відокремити рівень представлення інформації від рівня її обробки та зберігання. Це забезпечує кращу структурованість програмного коду, спрощує внесення змін у майбутньому, а також полегшує масштабування системи. Крім того, поділ на компоненти дозволяє зменшити зв'язність між частинами системи, що позитивно впливає на її підтримуваність і надійність.

У розробленому веб-сервісі можна виділити кілька основних архітектурних компонентів: користувацький інтерфейс, серверну логіку, модуль роботи з базою даних, підсистему пошуку та фільтрації, а також підсистему керування доступом. Кожен із цих компонентів виконує власну функцію, але всі вони працюють у тісному взаємозв'язку, забезпечуючи єдиний процес обробки даних та надання користувачам необхідного функціоналу.

Першим рівнем архітектури є клієнтський компонент, який реалізує інтерфейс взаємодії з користувачем. Саме через нього студент, викладач або адміністратор переглядає список дипломних проєктів, виконує пошук за темами, змінює статуси робіт, додає нові записи або переглядає аналітичну інформацію. Основним завданням цього компонента є забезпечення зручного, зрозумілого та швидкого доступу до функцій системи. Клієнтський компонент приймає введені користувачем дані, перетворює їх у запити до сервера та відображає відповідь у відповідному вигляді, наприклад у формі таблиці, списку, картки проєкту або графічної діаграми.

Другим важливим елементом є серверний компонент, який відповідає за обробку запитів, отриманих від клієнта. Саме тут реалізується основна логіка роботи застосунку. Сервер приймає запит, перевіряє його коректність, виконує необхідні операції над даними та формує відповідь для клієнтської частини.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Наприклад, якщо користувач надсилає запит на пошук дипломного проєкту за прізвищем студента або назвою теми, сервер отримує цей запит, аналізує передані параметри, виконує пошук у базі даних і повертає список відповідних результатів. Якщо ж користувач змінює статус проєкту, сервер перевіряє права доступу, оновлює запис у базі даних та повертає підтвердження успішного виконання операції.

Окрему роль у серверній частині виконує бізнес-логіка застосунку. Саме на цьому рівні реалізуються основні правила функціонування системи та перевіряється коректність виконання операцій користувачами. Наприклад, система може дозволяти студенту ініціювати зміну теми дипломного проєкту, однак остаточне підтвердження такої зміни повинно виконуватися науковим керівником або викладачем. Таким чином забезпечується контроль за коректністю змін та збереження академічного порядку в процесі закріплення тем. Крім того, на рівні бізнес-логіки перевіряються різні обмеження, зокрема можливість дублювання тем, відповідність студента певній групі чи курсу, правильність формату введених даних, а також наявність усіх необхідних полів під час створення або редагування записів. Завдяки такому підходу система не лише зберігає інформацію, але й забезпечує дотримання встановлених правил роботи та контролює коректність виконання основних процесів.

Наступним ключовим компонентом є база даних, що виконує функцію централізованого сховища інформації. У ній зберігаються дані про студентів, викладачів, теми дипломних проєктів, статуси виконання, строки здачі, прив'язки між темами та керівниками, а також інша службова інформація. База даних виступає єдиним джерелом істини для всієї системи, оскільки всі операції додавання, редагування, пошуку чи видалення інформації виконуються саме через неї. Правильна організація структури бази даних є критично важливою, оскільки від цього залежить швидкість пошуку, цілісність інформації та можливість масштабування застосунку.

Для забезпечення ефективної роботи база даних повинна підтримувати

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

логічні зв'язки між сутностями. Наприклад, кожен студент може бути пов'язаний із конкретною темою дипломного проєкту, а кожна тема - із відповідним науковим керівником. Крім того, проєкт може мати власний статус, дату створення, дедлайн та історію змін. Такий підхід дозволяє відображати складні взаємозалежності між елементами системи та виконувати запити різного рівня складності, наприклад знаходити всі дипломні проєкти певної групи, всі роботи конкретного керівника або список тем із певним статусом виконання.

Важливою підсистемою є модуль пошуку та фільтрації, оскільки саме пошук виступає однією з основних функцій веб-сервісу. Його завдання полягає у швидкому знаходженні потрібних записів за заданими критеріями. У межах архітектури цей модуль взаємодіє як з клієнтською частиною, так і з сервером та базою даних. Клієнт передає пошуковий запит і параметри фільтрації, сервер інтерпретує їх і формує відповідний запит до бази даних, після чого отримані результати повертаються користувачу. Такий механізм дозволяє здійснювати як простий пошук за ключовим словом, так і складну вибірку за кількома параметрами одночасно, наприклад за курсом, групою, керівником і статусом роботи.

Ще одним важливим компонентом архітектури веб-сервісу є підсистема керування доступом та автентифікації користувачів. Оскільки у системі взаємодіють різні категорії користувачів, необхідно реалізувати механізм розмежування прав доступу до функціоналу. Наприклад, адміністратор має повний доступ до управління записами та може створювати, редагувати або видаляти дані у системі. Викладач відповідає за перевірку та підтвердження інформації, пов'язаної з дипломними проєктами, а також може вносити зміни до закріплених за ним записів. Студент, у свою чергу, має можливість переглядати інформацію щодо власної дипломної роботи та ініціювати зміну теми проєкту, проте така зміна набуває чинності лише після підтвердження науковим керівником. Для реалізації такого підходу архітектура системи повинна передбачати механізми автентифікації користувачів, визначення їх ролей та подальший контроль доступу до відповідних функцій. Це дозволяє забезпечити безпечну роботу з даними та впорядкувати

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

процес взаємодії між учасниками навчального процесу.

Якщо розглядати взаємодію компонентів поетапно, то типовий сценарій роботи веб-сервісу виглядає так. Користувач відкриває веб-інтерфейс і виконує певну дію, наприклад вводить пошуковий запит або змінює статус дипломного проєкту. Клієнтська частина формує відповідний HTTP-запит і надсилає його на сервер. Серверний компонент приймає цей запит, перевіряє його коректність та права користувача, після чого звертається до бізнес-логіки. Бізнес-логіка визначає, які саме операції потрібно виконати, і за потреби формує запит до бази даних. База даних повертає результати або підтверджує виконання змін. Після цього сервер формує відповідь у форматі JSON і надсилає її назад клієнту. Клієнтська частина отримує дані та оновлює інтерфейс, відображаючи користувачу актуальну інформацію.

У межах даного проєкту архітектура може бути представлена як багаторівнева модель, у якій чітко розмежовані рівень представлення, рівень прикладної логіки та рівень даних. Рівень представлення відповідає за візуальне відображення інформації та взаємодію з користувачем. Рівень прикладної логіки обробляє бізнес-процеси, перевіряє дані та координує роботу інших компонентів. Рівень даних забезпечує фізичне зберігання та вибірку інформації. Така організація позитивно впливає на модульність системи, полегшує тестування та дає змогу розширювати окремі компоненти без повного перепроєктування всього застосунку.

Додатково архітектура веб-сервісу повинна враховувати можливість подальшого розширення функціоналу. Наприклад, у майбутньому до системи можуть бути додані модулі сповіщень, автоматичного формування звітів, інтеграції з електронним журналом, експорту документів або розширеної аналітики. Якщо архітектура спочатку побудована правильно, такі зміни можна реалізувати без істотного втручання в уже наявний функціонал. Саме тому важливо забезпечити низьку зв'язаність між модулями, використання уніфікованих інтерфейсів взаємодії та чіткий розподіл відповідальності між компонентами.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

У висновку, архітектура веб-сервісу для моніторингу дипломних проєктів є багатокомпонентною системою, у якій кожен елемент виконує окрему роль, але всі вони працюють узгоджено. Клієнтський інтерфейс забезпечує взаємодію з користувачем, серверна частина реалізує обробку запитів і бізнес-логіку, база даних відповідає за централізоване зберігання інформації, а підсистеми пошуку, автентифікації та аналітики розширюють функціональні можливості системи. Такий підхід забезпечує структурованість, гнучкість, надійність та зручність використання веб-застосунку, що є необхідною умовою для його ефективного застосування в освітньому середовищі.

2.4 Вибір технологій для розробки веб-сервісу

Під час розробки веб-сервісу для моніторингу дипломних проєктів важливим етапом є вибір технологій, які забезпечать стабільну роботу системи, зручність підтримки, можливість подальшого розширення функціоналу та інтеграцію з іншими інформаційними ресурсами навчального закладу. Оскільки розроблюваний застосунок орієнтований не лише на демонстрацію в межах дипломного проєкту, а й потенційно може бути впроваджений у реальне середовище закладу вищої освіти, доцільно використовувати сучасний і масштабований технологічний стек.

Для реалізації клієнтської частини веб-сервісу було обрано React. Дана бібліотека дозволяє будувати динамічні користувацькі інтерфейси на основі компонентного підходу. Така організація коду дає змогу розділяти інтерфейс на окремі незалежні елементи, що значно спрощує розробку, повторне використання компонентів та подальшу підтримку застосунку. Для системи моніторингу дипломних проєктів це є особливо важливим, оскільки інтерфейс повинен містити таблиці, форми введення, засоби фільтрації, сторінки перегляду інформації про студентів, керівників і теми, а також аналітичні елементи.

Як мову програмування для клієнтської частини доцільно використовувати

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

TypeScript, який є розширенням JavaScript та забезпечує статичну типізацію. Використання TypeScript дозволяє підвищити якість коду, зменшити кількість помилок під час розробки та зробити структуру програми більш зрозумілою. Для системи, що працює з великою кількістю взаємопов'язаних сутностей, таких як студенти, викладачі, дипломні проєкти, статуси виконання та строки здачі, типізація є важливою перевагою, оскільки забезпечує контроль коректності даних на етапі програмування.

Для стилізації інтерфейсу було обрано Tailwind CSS. Цей інструмент дозволяє швидко створювати сучасний та адаптивний інтерфейс за допомогою набору утилітарних класів. Перевагою такого підходу є зменшення кількості окремих CSS-файлів, спрощення підтримки стилів та можливість швидко змінювати зовнішній вигляд компонентів. Адаптивність інтерфейсу є важливою характеристикою для розроблюваної системи, оскільки користувачі можуть працювати з нею з різних пристроїв, зокрема з персональних комп'ютерів, ноутбуків та планшетів.

Для реалізації серверної частини було обрано NestJS. Даний фреймворк для Node.js є одним із найбільш придатних інструментів для побудови структурованих, масштабованих та надійних серверних застосунків. NestJS побудований із використанням TypeScript та підтримує модульну архітектуру, що дозволяє логічно розділити серверну частину на окремі функціональні блоки, наприклад модулі керування студентами, дипломними проєктами, викладачами, автентифікацією та пошуком. Такий підхід забезпечує зрозумілу структуру проєкту, полегшує супровід програмного коду та створює основу для подальшого розширення системи.

Однією з причин вибору NestJS є його орієнтація на розробку повноцінного окремого бекенду, що є доцільним у випадку можливого подальшого впровадження веб-сервісу в інфраструктуру навчального закладу. Використання окремого серверного API дозволяє відокремити клієнтську частину від серверної логіки, що спрощує інтеграцію системи з іншими веб-ресурсами, внутрішніми сервісами університету або зовнішніми платформами. Крім того, NestJS надає зручні засоби для реалізації REST API, перевірки вхідних даних, розмежування доступу між

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

ролями користувачів та побудови безпечної серверної логіки.

Для роботи з базою даних було обрано PostgreSQL. Дана система керування базами даних є надійною, продуктивною та добре підходить для серверних застосунків, у яких необхідно зберігати структуровану інформацію та підтримувати складні зв'язки між сутностями. У межах розроблюваного веб-сервісу база даних повинна зберігати інформацію про студентів, викладачів, теми дипломних проєктів, статуси виконання, строки здачі, історію змін та інші дані. PostgreSQL забезпечує підтримку реляційної моделі, транзакцій, обмежень цілісності та багатокористувацького доступу, що робить її доцільним вибором для подібної системи.

Для взаємодії між серверною частиною та базою даних доцільно використовувати Prisma ORM. Prisma виконує роль прошарку між прикладним кодом і базою даних, дозволяючи працювати із записами через моделі та програмний інтерфейс замість написання великої кількості ручних SQL-запитів. Це зменшує ризик помилок, полегшує підтримку коду та прискорює розробку. Окрім цього, Prisma спрощує опис структури бази даних, створення міграцій і контроль зв'язків між сутностями, що є важливим для системи з кількома пов'язаними таблицями.

Для імпорту даних із файлів електронних таблиць використовується бібліотека `xlsx`. Її застосування дозволяє зчитувати дані з Excel-файлів та перетворювати їх у формат, придатний для подальшої обробки на сервері та збереження у базі даних. Такий функціонал є необхідним для первинного заповнення системи або масового оновлення інформації про студентів, теми та викладачів без ручного введення кожного запису.

Для реалізації нечіткого пошуку може використовуватися `Fuse.js`. Ця бібліотека забезпечує пошук за частковими або неточними збігами, що підвищує зручність роботи користувачів із системою. Наприклад, якщо користувач вводить лише частину назви теми або прізвища студента, система все одно може знайти релевантні результати. Використання такого механізму є доцільним у тих випадках,

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

коли необхідно забезпечити швидкий і гнучкий пошук по невеликих або середніх наборах уже отриманих даних. Якщо ж у подальшому обсяг даних суттєво зростатиме, пошукову логіку можна буде перенести безпосередньо на серверний рівень.

Для візуального оформлення інтерфейсу та покращення навігації можуть використовуватися іконки з бібліотеки `lucide-react`. Їх застосування дає змогу зробити інтерфейс більш зрозумілим для користувача, акцентувати увагу на окремих діях та покращити загальне сприйняття елементів керування.

Для організації безпечного доступу до функцій системи на серверному рівні в `NestJS` доцільно використовувати механізми автентифікації та авторизації на основі `JWT`. Це дозволяє перевіряти особу користувача під час входу в систему, визначати його роль та надавати доступ лише до тих функцій, які відповідають його повноваженням. Наприклад, адміністратор може керувати всіма записами, викладач - підтверджувати зміни тем і редагувати пов'язані з ним дані, а студент - переглядати власну інформацію та ініціювати запити на зміну теми. Такий підхід забезпечує захист персональних даних і впорядковує взаємодію між учасниками системи.

Для обміну даними між клієнтською та серверною частинами використовується формат `JSON`, а взаємодія здійснюється за допомогою `HTTP`-запитів до `REST API`. Така модель є стандартною для сучасних веб-застосунків і забезпечує сумісність між різними компонентами системи. Вона також спрощує подальшу інтеграцію з іншими клієнтами, якщо в майбутньому виникне потреба розробки мобільного застосунку або підключення додаткової адміністративної панелі.

Під час організації процесу розробки доцільно використовувати **Git** як систему контролю версій. Це дозволяє зберігати історію змін, контролювати розвиток проєкту, повертатися до попередніх версій у разі необхідності та підтримувати більш структурований підхід до розробки.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Отже, обраний технологічний стек, що включає React, TypeScript, Tailwind CSS, NestJS, PostgreSQL, Prisma, xlsx, Fuse.js та lucide-react, є доцільним для створення веб-сервісу моніторингу дипломних проєктів. Він забезпечує достатню гнучкість, модульність, зручність підтримки та можливість подальшого масштабування системи. Використання окремого бекенду на базі NestJS додатково підвищує перспективність такого рішення в умовах можливого подальшого впровадження та інтеграції з інформаційними системами навчального закладу.

2.5 Безпека та інтеграція компонентів веб-сервісу

Під час розробки важливим аспектом є забезпечення безпеки даних та коректної взаємодії між усіма компонентами системи. Оскільки застосунок працює з інформацією про студентів, викладачів та теми дипломних робіт, необхідно гарантувати захист персональних даних, контроль доступу до функцій системи та стабільну передачу інформації між її складовими.

Одним із ключових елементів безпеки є контроль доступу до системи. Користувачі повинні проходити процедуру аутентифікації, що дозволяє ідентифікувати їх у системі та визначити рівень доступу до функціоналу. Залежно від ролі користувача (адміністратор, викладач або студент) система надає різні права доступу до даних і операцій. Такий підхід дозволяє обмежити можливість несанкціонованого редагування або видалення інформації та забезпечує відповідальність користувачів за внесені зміни.

Для захисту передачі даних між клієнтською частиною та сервером використовується захищений протокол HTTPS. Його застосування забезпечує шифрування інформації під час обміну даними, що унеможливорює перехоплення або модифікацію даних сторонніми особами. Крім того, важливою складовою безпеки є перевірка коректності вхідних даних. Серверна частина системи повинна здійснювати валідацію запитів користувачів, щоб запобігти помилкам введення та можливим спробам несанкціонованого доступу до бази даних.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Іншим важливим аспектом є збереження цілісності інформації. Система повинна гарантувати, що всі зміни у базі даних виконуються контрольовано та не призводять до порушення зв'язків між записами. Наприклад, при зміні або видаленні теми дипломного проєкту необхідно враховувати наявність пов'язаних записів про студентів або керівників. Для цього використовуються механізми контролю цілісності даних на рівні бази даних і серверної логіки.

Не менш важливою є інтеграція компонентів веб-сервісу. Система побудована за клієнт-серверною архітектурою, у якій окремі модулі виконують визначені функції. Клієнтська частина відповідає за інтерфейс користувача, відображення інформації та формування запитів до серверу. Серверна частина забезпечує обробку цих запитів, виконання логіки роботи застосунку та взаємодію з базою даних. Такий підхід дозволяє розділити відповідальність між компонентами системи та підвищує її масштабованість і гнучкість.

Обмін даними між клієнтом і сервером відбувається через API-інтерфейс. Кожен запит користувача передається у вигляді HTTP-запиту до відповідного ендпоінту, після чого сервер обробляє його та повертає результат у структурованому форматі, наприклад JSON. Це дозволяє стандартизувати взаємодію між компонентами системи та спрощує інтеграцію з іншими сервісами.

Завдяки такій архітектурі система може бути легко розширена або інтегрована з іншими інформаційними системами навчального закладу. Наприклад, веб-застосунок може взаємодіяти з електронними журналами, системами дистанційного навчання або внутрішніми базами даних університету. Це створює єдине інформаційне середовище, у якому дані можуть використовуватися різними підрозділами без дублювання.

Таким чином, забезпечення безпеки та налагоджена інтеграція компонентів веб-сервісу є важливими умовами стабільної роботи системи. Використання механізмів контролю доступу, шифрування даних, перевірки вхідної інформації та клієнт-серверної архітектури дозволяє створити надійний і масштабований веб-застосунок.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

3 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ ПОШУКУ ТА ОБЛІКУ ПРОЄКТІВ

3.1 Загальна архітектура системи

На основі розроблених у попередньому розділі моделей і алгоритмів реалізовано програмну систему моніторингу дипломних проєктів університетської кафедри. Система побудована за клієнт-серверною архітектурою та складається з двох основних частин: клієнтського застосунку та серверного застосунку, які взаємодіють між собою через REST API.

Клієнтська частина розроблена з використанням Next.js 16, React 19 та TypeScript. Використання Next.js дозволяє реалізувати серверний рендеринг, оптимізувати завантаження сторінок і зручно організувати маршрутизацію. Клієнтський застосунок відповідає за відображення даних, взаємодію з користувачем та надсилання запитів до серверної частини.

Серверна частина реалізована на основі NestJS 10. Даний фреймворк забезпечує модульну організацію застосунку, підтримку контролерів, сервісів, ін'єкції залежностей та засобів обробки запитів. Сервер виконує основну бізнес-логіку системи, обробляє запити клієнта, взаємодіє з базою даних і формує відповіді API.

Обмін даними між frontend і backend здійснюється через протокол HTTP з використанням REST-підходу. Клієнтський застосунок працює на порту 3000, серверний — на порту 3001. Для безпечної взаємодії між компонентами на стороні сервера налаштовано CORS, що дозволяє приймати запити лише з визначеного джерела.

Серверна частина має модульну структуру і включає модуль пошуку, модуль фільтрів та модуль завантаження. Для доступу до бази даних використовується Prisma ORM, а зберігання даних реалізовано за допомогою SQLite у файлі dev.db.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

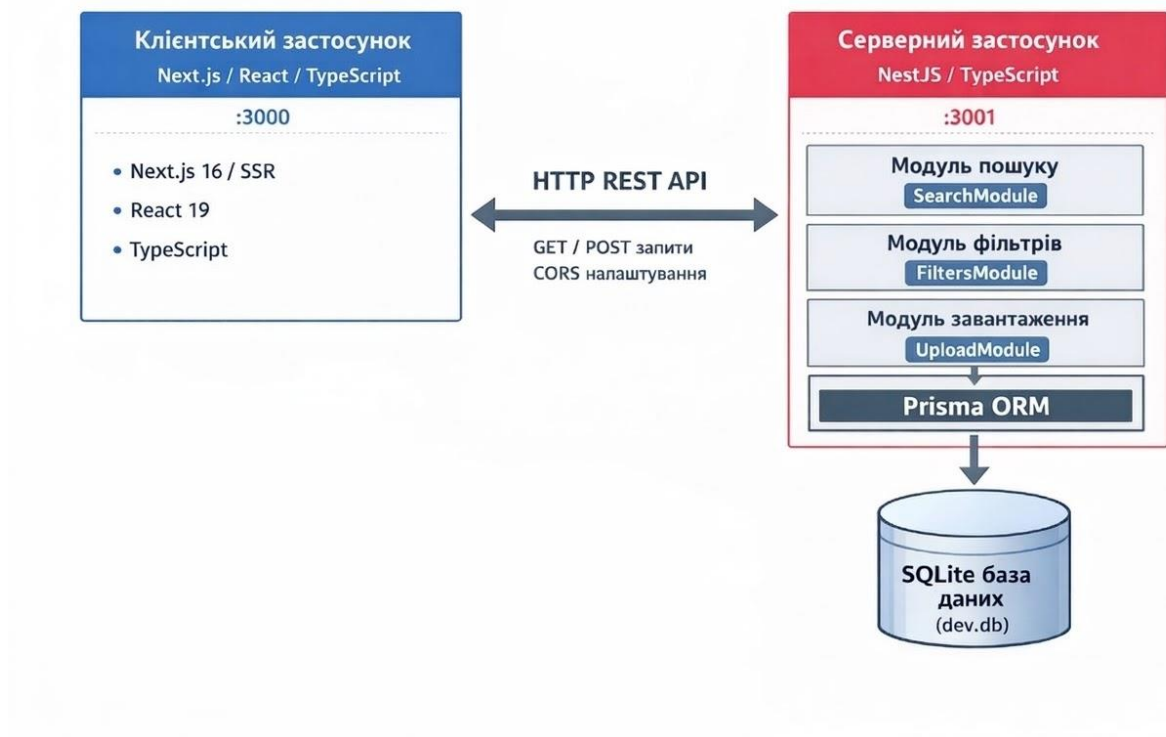


Рисунок 3.1 - Загальна архітектура системи моніторингу дипломних проєктів

Отже, обрана архітектура забезпечує чіткий розподіл функцій між інтерфейсом користувача, серверною логікою та рівнем зберігання даних. Це спрощує розробку, підтримку та подальше розширення системи.

3.2 Проєктування та реалізація бази даних

Проєктування бази даних є важливим етапом розробки інформаційної системи, оскільки саме структура збереження даних визначає цілісність, зручність обробки та ефективність подальшої роботи застосунку. У межах розробленої системи було визначено основні сутності предметної області, зв'язки між ними та засоби програмної взаємодії з базою даних. Для реалізації цього компонента необхідно було обрати відповідну систему управління базами даних і ORM-засіб, а також сформулювати логічну та фізичну схему збереження інформації.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

3.2.1 Вибір системи управління базою даних та ORM

Для зберігання інформації про студентів, викладачів і дипломні проекти у розробленій системі обрано реляційну систему управління базами даних SQLite. Такий вибір зумовлений особливостями предметної області та масштабом навантаження, на який орієнтований програмний застосунок. Оскільки система призначена для використання на рівні університетської кафедри, очікуваний обсяг даних є відносно невеликим, тому застосування повноцінної серверної СУБД у цьому випадку не є обов'язковим.

Однією з головних переваг SQLite є простота розгортання. На відміну від серверних систем керування базами даних, вона не потребує окремого процесу обслуговування, налаштування мережевого підключення чи адміністрування сервера баз даних. Уся база зберігається у вигляді одного файлу dev.db, що спрощує запуск і перенесення системи між різними середовищами. Це особливо зручно для навчального або кафедрального програмного продукту, де важливими є швидкість налаштування, мінімальні вимоги до інфраструктури та простота супроводу.

Ще однією важливою причиною вибору SQLite є її достатня продуктивність для обробки навантаження кафедрального масштабу. Система передбачає роботу з сотнями або тисячами записів, що є цілком прийнятним сценарієм для такої СУБД. Крім того, SQLite підтримує транзакційний механізм і властивості ACID, що дозволяє забезпечити цілісність, узгодженість і надійність збереження даних під час виконання операцій створення, оновлення та видалення записів.

Для організації взаємодії серверної частини з базою даних було обрано Prisma ORM. Використання ORM дає змогу працювати з даними не на рівні сирих SQL-запитів, а через описані у схемі моделі та типізований програмний інтерфейс. Це спрощує розробку серверної логіки, покращує читабельність коду та зменшує ймовірність помилок при роботі з таблицями та зв'язками між ними.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Prisma використовує файл `schema.prisma`, у якому описуються моделі бази даних, їхні поля, типи, обмеження та зв'язки. На основі цієї схеми автоматично генерується типобезпечний клієнт для TypeScript, що дозволяє зручно виконувати операції над даними без ручного опису більшості низькорівневих деталей. Важливою перевагою Prisma також є підтримка механізму міграцій, завдяки якому структура бази даних може змінюватися та розширюватися в процесі розвитку системи без потреби вручну переписувати SQL-сценарії.

Отже, поєднання SQLite та Prisma ORM є доцільним для даної системи, оскільки забезпечує простоту розгортання, достатню продуктивність, надійність збереження даних та зручність програмної взаємодії з базою.

3.2.2 Схема бази даних

Схема бази даних реалізована у файлі `prisma/schema.prisma` та побудована відповідно до логічної моделі предметної області. Основними сутностями системи є `Student`, `Teacher` та `Project`. Вони відображають ключові об'єкти, з якими працює система моніторингу дипломних проєктів, а саме студентів, наукових керівників та самі дипломні проєкти.

Модель `Student` призначена для зберігання інформації про студентів. Вона містить ідентифікатор запису, прізвище, ім'я та по батькові студента, а також додаткові атрибути, такі як академічна група та курс навчання. Частина полів є необов'язковою, що дозволяє зберігати записи навіть у разі неповного набору вхідних даних.

```
model Student {
  id      String  @id @default(uuid())
  name    String  // ПІБ студента
  group   String? // Група (опціонально)
  course  String? // Курс студента
  project Project?
}
```

Рисунок 3.2.2 – Модель таблиці студента

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Модель Teacher зберігає інформацію про викладачів або наукових керівників. Вона включає унікальний ідентифікатор та ім'я викладача. На відміну від моделі студента, у даному випадку передбачено зв'язок типу один-до-багатьох із моделлю Project, оскільки один викладач може керувати кількома дипломними проєктами одночасно.

```
model Teacher {
  id      String  @id @default(uuid())
  name    String  // ПІБ викладача/керівника
  projects Project[] // Проєкти, якими керує
}
```

Рисунок 3.2.2 – модель таблиці викладача

Центральною сутністю всієї схеми є модель Project, оскільки саме вона об'єднує основну інформацію про дипломну роботу. У цій моделі зберігаються тема проєкту, опис, поточний статус виконання, семестр або інша додаткова класифікаційна ознака, а також зовнішні ключі, що пов'язують проєкт із конкретним студентом і викладачем. Крім того, модель містить службові поля createdAt і updatedAt, які дозволяють автоматично фіксувати час створення запису та останнього оновлення. Це є корисним для моніторингу змін і подальшого аудиту даних у системі.

```
model Project {
  id      String  @id @default(uuid())
  title    String  // Тема дипломного проєкту
  description String? // Додаткова інформація
  status   String  @default("В процесі") // Статус (напр. Захищено, В процесі)
  semester String? // Семестр/дисципліна (назва аркуша Excel)

  studentId String  @unique
  student   Student @relation(fields: [studentId], references: [id], onDelete: Cascade)

  teacherId String
  teacher   Teacher @relation(fields: [teacherId], references: [id], onDelete: Restrict)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Рисунок 3.2.2 – модель таблиці проєктів

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Між моделями реалізовано чітко визначені зв'язки. Між Student і Project використовується відношення один-до-одного, що реалізується через унікальне поле studentId у моделі Project. Такий підхід означає, що кожен студент може бути пов'язаний лише з одним активним дипломним проєктом. Це відповідає предметній області системи, оскільки в межах одного періоду студент, як правило, виконує лише одну дипломну роботу.

Між Teacher і Project реалізовано відношення один-до-багатьох. Це означає, що один науковий керівник може мати декілька закріплених за ним дипломних проєктів, тоді як кожен окремий проєкт має лише одного відповідального керівника. Така структура дозволяє коректно моделювати типову організацію навчального процесу на кафедрі.

Окрему увагу приділено правилам цілісності даних під час видалення пов'язаних записів. Для зв'язку між студентом і проєктом використано правило onDelete: Cascade, згідно з яким при видаленні запису про студента автоматично видаляється й пов'язаний із ним проєкт. Це дозволяє уникнути появи в базі даних «висячих» записів, які втратили логічний зв'язок із основною сутністю. Для зв'язку між викладачем і проєктом застосовано правило onDelete: Restrict, що забороняє видалення викладача, якщо в базі залишаються пов'язані з ним проєкти. Таке обмеження забезпечує збереження посилальної цілісності та запобігає втраті важливої інформації про наукове керівництво дипломними роботами.

Усі основні сутності системи використовують як первинний ключ ідентифікатори типу UUID, що генеруються автоматично. Використання UUID забезпечує глобальну унікальність записів, спрощує їх ідентифікацію в межах системи та зменшує ризик конфліктів ключів. Додатковою перевагою такого підходу є менша передбачуваність ідентифікаторів у порівнянні з послідовними числовими ключами.

Таким чином, реалізована схема бази даних охоплює основні об'єкти предметної області та зв'язки між ними, забезпечує цілісність і структурованість

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

даних, а також створює надійну основу для подальшої роботи всієї інформаційної системи.

3.3 Реалізація серверної частини

Серверна частина системи реалізована на основі фреймворку NestJS, який забезпечує модульну організацію застосунку, чіткий розподіл відповідальності між компонентами та зручні засоби для побудови REST API. Використання NestJS дозволило структуровано організувати серверну логіку системи, виділити окремі функціональні модулі та забезпечити зручну взаємодію з базою даних через механізм ін'єкції залежностей.

Серверний застосунок виконує основні функції обробки даних, взаємодії з базою даних, реалізації пошуку, фільтрації та імпорту інформації з зовнішніх джерел. Архітектура backend-частини побудована таким чином, щоб окремі функціональні можливості були ізольовані в незалежних модулях, що спрощує підтримку, тестування та подальше розширення системи.

3.3.1 Структура та конфігурація серверного застосунку

Серверний застосунок побудований за модульним принципом, який є базовим підходом у NestJS. Центральним елементом структури є кореневий модуль AppModule, що об'єднує всі функціональні підмодулі системи. До його складу входять PrismaModule, FiltersModule, SearchModule та UploadModule. Така організація дозволяє розділити логіку застосунку на окремі змістовні частини, кожна з яких відповідає за власний набір функцій.

```
@Module({
  imports: [PrismaModule, FiltersModule, SearchModule, UploadModule],
})
export class AppModule { }
```

Рисунок 3.3.1 - Кореневий модуль серверного застосунку

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

PrismaModule виконує роль спільного модуля доступу до бази даних. Через нього іншим модулям надається єдиний екземпляр PrismaService, який використовується для виконання операцій над даними. Завдяки механізму ін'єкції залежностей NestJS цей сервіс може повторно використовуватися в різних частинах системи без дублювання логіки підключення до бази даних.

Точка входу до серверного застосунку реалізована у файлі main.ts. На цьому етапі створюється екземпляр NestJS-застосунку, налаштовується підтримка CORS та запускається HTTP-сервер. Сервер працює на порту 3001, тоді як клієнтська частина — на порту 3000, тому для коректної взаємодії між ними було дозволено запити лише з конкретного джерела `http://localhost:3000`. Таке обмеження є стандартною практикою захисту REST API, оскільки дозволяє уникнути небажаних запитів із неавторизованих джерел.

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.enableCors({
    origin: 'http://localhost:3000',
    methods: ['GET', 'POST'],
  });
  await app.listen(3001);
  console.log('NestJS backend running on http://localhost:3001');
}
bootstrap();
```

Рисунок 3.3.1 - Точка входу main.ts

Отже, структура серверної частини є модульною, логічно впорядкованою та орієнтованою на подальше масштабування. Поділ на окремі модулі забезпечує зрозумілу організацію програмного коду та спрощує реалізацію окремих функціональних підсистем.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

3.3.2 Модуль імпорту даних

Модуль UploadModule призначений для обробки завантажених Excel-файлів формату .xlsx та .xls, які містять інформацію про курсові та кваліфікаційні роботи кафедри. Основним завданням цього модуля є приймання файлу від користувача, його аналіз, вилучення структурованих даних та збереження цих даних до бази.

```
@Controller('upload')
export class UploadController {
  constructor(private readonly uploadService: UploadService) { }

  @Post()
  @UseInterceptors(FileInterceptor('file', { storage: memoryStorage() }))
  async upload(@UploadedFile() file: Express.Multer.File) {
    if (!file) {
      throw new BadRequestException('Файл не знайдено');
    }

    const name = file.originalname;

    if (name.endsWith('.xlsx') || name.endsWith('.xls')) {
      try {
        return await this.uploadService.handleExcelUpload(file.buffer);
      } catch (error) {
        console.error('Excel parse error:', error);
        throw new InternalServerErrorException(
          'Помилка парсингу Excel файлу. Перевірте структуру.',
        );
      }
    } else if (name.endsWith('.pdf')) {
      throw new BadRequestException(
        'Обробка PDF ще в процесі розробки. Використовуйте Excel.',
      );
    } else {
      throw new BadRequestException('Непідтримуваний формат файлу');
    }
  }
}
```

Рисунок 3.3.2 - Контролер UploadController

Взаємодія з модулем здійснюється через маршрут POST /upload. У контролері UploadController для приймання файлу використовується FileInterceptor із бібліотеки @nestjs/platform-express у поєднанні з multer у режимі memoryStorage(). Це означає, що файл не записується на диск, а обробляється безпосередньо в оперативній пам'яті у вигляді буфера. Такий підхід спрощує реалізацію завантаження та є доцільним для відносно невеликих файлів, характерних для кафедральної документації.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Після отримання файлу виконується базова перевірка його наявності та формату. Якщо файл не передано, генерується виняток `BadRequestException`. Якщо формат відповідає Excel-файлу, керування передається до сервісу `UploadService`, який реалізує основний алгоритм імпорту.

Для аналізу вмісту файлу використовується бібліотека `xlsx`. На першому етапі виконується зчитування книги з буфера, після чого послідовно обробляються всі аркуші документа. Кожен аркуш інтерпретується як окремий семестр або дисципліна, а його назва зберігається у полі `semester`. Такий підхід дозволяє автоматично прив'язати імпортовані записи до відповідного контексту навчального процесу.

У процесі обробки даних сервіс аналізує вміст рядків аркуша та визначає поточне значення курсу на основі текстових шаблонів, наприклад «3 курс» або «1 курс магістратури». Отримане значення використовується як резервне, якщо в конкретному рядку відсутня явна інформація про групу. Далі кожен рядок таблиці перетворюється на масив значень за допомогою `sheet_to_json(..., { header: 1 })`, після чого з нього зчитуються потрібні колонки: прізвище, ім'я та по батькові студента, тема проєкту, прізвище керівника та, за наявності, назва академічної групи.

У процесі обробки даних сервіс аналізує вміст рядків аркуша та визначає поточне значення курсу на основі текстових шаблонів, наприклад «3 курс» або «1 курс магістратури». Отримане значення використовується як резервне, якщо в конкретному рядку відсутня явна інформація про групу. Далі кожен рядок таблиці перетворюється на масив значень за допомогою `sheet_to_json(..., { header: 1 })`, після чого з нього зчитуються потрібні колонки: прізвище, ім'я та по батькові студента, тема проєкту, прізвище керівника та, за наявності, назва академічної групи.

Окрему увагу приділено валідації вхідних даних. Із загального набору рядків відфільтровуються службові елементи, заголовки таблиць і порожні записи. Також перевіряється коректність ПІБ студента, зокрема мінімальна довжина рядка та наявність щонайменше двох слів. Це дозволяє зменшити кількість помилкових або шумових записів, які можуть потрапити до бази даних під час імпорту.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Збереження даних реалізоване за принципом «знайти або створити». Для студентів і викладачів спочатку виконується пошук запису за ім'ям. Якщо відповідна сутність уже існує, вона повторно використовується; якщо ні — створюється новий запис. Для студента, за потреби, може оновлюватися інформація про групу. Для дипломного проєкту перевіряється наявність уже існуючого запису, пов'язаного з конкретним студентом. Якщо такий запис знайдено, виконується його оновлення; якщо ні — створюється новий.

Важливою властивістю реалізованого підходу є ідемпотентність. Повторне завантаження одного й того самого файлу не призводить до дублювання записів у базі даних, а лише актуалізує вже наявну інформацію. Це підвищує надійність роботи модуля та спрощує повторний імпорт даних у разі їх оновлення.

3.3.3 Модуль нечіткого пошуку

Модуль SearchModule реалізує механізм нечіткого пошуку дипломних проєктів за текстовим запитом користувача. Його основним призначенням є пошук релевантних записів навіть у тих випадках, коли запит містить неточності, неповні формулювання або часткові збіги. Доступ до функціональності модуля здійснюється через маршрут GET /search із параметрами запиту, зокрема текстом пошуку, викладачем і семестром.

Алгоритм пошуку реалізований у SearchService і поєднує кілька етапів обробки даних. На першому етапі виконується попередня фільтрація через Prisma. Якщо користувач задає додаткові обмеження, наприклад конкретного викладача або семестр, система спочатку звужує вибірку проєктів у базі даних за відповідними умовами. Це дозволяє зменшити кількість записів, над якими надалі виконується нечіткий пошук, і таким чином підвищити ефективність роботи алгоритму.

Наступним етапом є токенізація текстового запиту. Введений користувачем рядок розбивається на окремі слова, після чого з нього вилучаються службові слова та короткі фрагменти, які не несуть суттєвого змістового навантаження. До таких

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

слів належать, наприклад, прийменники, сполучники та частки. Подібне очищення запиту дозволяє зменшити кількість шуму та підвищити якість пошуку.

Основний механізм пошуку реалізовано за допомогою бібліотеки Fuse.js, яка підтримує нечітке порівняння текстових рядків. Пошук виконується окремо для кожного значущого слова запиту по полях title і student.name. Серед важливих параметрів налаштування використовується ignoreLocation: true, що дозволяє знаходити збіги в будь-якій частині текстового поля, а не лише на його початку. Це є особливо корисним для пошуку за ключовими словами у формулюваннях тем дипломних робіт.

Після отримання часткових результатів відбувається їх агрегація та ранжування. Для кожного знайденого проєкту накопичується інформація про кількість збігів із токенами запиту та сумарну оцінку схожості. Першочерговий пріоритет надається точним збігам фрази, далі враховується кількість збігів за словами запиту, а потім — значення нечіткого коефіцієнта. Такий підхід дозволяє впорядкувати результати за релевантністю та вивести користувачеві найбільш доречні записи.

Для зменшення кількості випадкових збігів у модулі реалізовано поріг релевантності. Проєкт включається до результатів лише тоді, коли збіги знайдено для більшості слів запиту. Якщо за основним порогом результатів не виявлено, застосовується більш м'який резервний поріг. Крім того, кількість повернутих результатів обмежується першими 100 записами, що запобігає перевантаженню відповіді та зберігає зручність роботи користувача з результатами пошуку.

Отже, модуль нечіткого пошуку забезпечує більш гнучку та зручну роботу з базою дипломних проєктів, ніж звичайний точний пошук, оскільки дозволяє знаходити інформацію навіть за неточними або неповними запитами.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

3.3.4 Модуль фільтрів

Модуль `FiltersModule` призначений для формування та надання клієнтській частині актуальних значень фільтрів, які можуть бути використані для уточнення вибірки дипломних проєктів. Доступ до функціональності цього модуля здійснюється через маршрут `GET /filters`.

Основне завдання модуля полягає у формуванні списків доступних семестрів і викладачів на основі наявних записів у базі даних. Для цього сервіс `FiltersService` виконує запит через `Prisma`, у межах якого вибираються лише необхідні поля — `semester` і `teacher.name`. Після цього з отриманих значень формуються множини унікальних елементів, які перетворюються у відсортовані масиви та повертаються клієнтському застосунку.

Ключовою особливістю модуля є підтримка залежних фільтрів. Якщо користувач уже обрав одне із значень, наприклад семестр, у відповідь система повертає лише тих викладачів, які мають пов'язані проєкти в межах цього семестру. Аналогічно, якщо обрано викладача, список доступних семестрів автоматично звужується до тих, у яких для цього викладача існують записи. Такий механізм покращує зручність користування системою, оскільки користувачеві не пропонуються значення фільтрів, що фактично не мають результатів.

Таким чином, модуль фільтрів забезпечує динамічне формування доступних параметрів відбору та підвищує ефективність навігації по даних у системі.

3.4 Реалізація клієнтської частини (Next.js Frontend)

Клієнтська частина системи реалізована як вебзастосунок на основі `Next.js` з використанням `React` і `TypeScript`. Основне призначення frontend-частини полягає у забезпеченні зручної взаємодії користувача із системою, відображенні даних про дипломні проєкти, реалізації пошуку, фільтрації та взаємодії з серверною частиною

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

через REST API. Архітектура клієнтського застосунку побудована таким чином, щоб забезпечити швидке оновлення інтерфейсу, мінімізувати затримки під час пошуку та зробити роботу користувача з даними максимально зручною.

3.4.1 Структура клієнтського застосунку

Клієнтська частина побудована як застосунок Next.js App Router. Основна логіка інтерфейсу зосереджена на головній сторінці `src/app/page.tsx`, яка реалізована як клієнтський React-компонент із директивою `"use client"`. Це дозволяє використовувати повноцінне керування станом на стороні браузера, обробку подій користувача та динамічне оновлення інтерфейсу без необхідності переходу між додатковими маршрутами для базового функціоналу системи.

```
type ProjectResult = {
  item: {
    id: string;
    title: string;
    description: string | null;
    status: string;
    semester: string | null;
    student: { name: string; group: string | null };
    teacher: { name: string };
  };
  matches?: any[];
};
```

Рисунок 3.4.1 - Оголошені типи TypeScript для структур даних

Фактично головна сторінка реалізує єдиний SPA-подібний екран, на якому зосереджено основні можливості: введення пошукового запиту, вибір фільтрів, перегляд результатів та взаємодія з даними. Такий підхід спрощує навігацію та підвищує зручність використання системи, оскільки користувач працює в межах одного цілісного інтерфейсу.

Керування станом застосунку виконується за допомогою стандартних React-хуків, зокрема `useState`, `useEffect` та `useMemo`. `useState` використовується для збереження поточних значень пошукового рядка, вибраних фільтрів, отриманих

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

результатів і допоміжних службових станів. `useEffect` забезпечує виконання побічних операцій, наприклад надсилання запитів до сервера при зміні параметрів пошуку. `useMemo` застосовується для оптимізації обчислень і повторного використання вже підготовлених результатів без зайвих перерахунків під час кожного рендерингу компонента.

Для опису даних, що надходять із серверної частини, використано типи TypeScript. Це забезпечує типобезпечність при роботі з API, підвищує надійність клієнтського коду та дозволяє уникати багатьох помилок ще на етапі компіляції. Зокрема, для результатів пошуку визначено окрему типізовану структуру, яка містить інформацію про проєкт, студента, наукового керівника та додаткові дані про збіги.

3.4.2 Механізм дебаунсингу пошукових запитів

Для зменшення навантаження на сервер та уникнення надсилання великої кількості запитів під час введення тексту реалізовано механізм **дебаунсингу** пошукового запиту. Його суть полягає в тому, що фактичний запит до сервера виконується не після кожного натискання клавіші, а лише після короткої паузи у введенні.

У реалізації для цього використовуються два окремі стани: `query`, який відображає поточне значення поля вводу, та `debouncedQuery`, який оновлюється із затримкою 300 мілісекунд. При кожній зміні тексту запускається таймер. Якщо до завершення цього часу користувач вводить новий символ, попередній таймер скасовується і запускається новий.

```
useEffect(() => {
  const timer = setTimeout(() => {
    setDebouncedQuery(query);
  }, 300);

  return () => clearTimeout(timer);
}, [query]);
```

Рисунок 3.4.2 - Дебаунсинг пошукових запитів

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Такий підхід суттєво зменшує кількість API-запитів, покращує продуктивність системи та робить взаємодію з пошуком плавнішою. Особливо це важливо в ситуаціях, коли пошук виконується в реальному часі та супроводжується динамічним оновленням результатів.

3.4.3 Клієнтська реалізація нечіткого пошуку

Окрім серверного механізму пошуку, у клієнтській частині реалізовано додаткову локальну фільтрацію результатів із використанням алгоритму Левенштейна. Це дозволяє уточнювати вже отримані з сервера дані та покращувати якість відображення результатів у випадках, коли введений користувачем текст містить друкарські помилки або часткові розбіжності з текстом у базі даних.

Алгоритм Левенштейна реалізовано безпосередньо мовою TypeScript. Він обчислює мінімальну кількість операцій вставки, видалення та заміни символів, необхідних для перетворення одного рядка в інший. Для зменшення обчислювальної складності використано оптимізацію дострокового завершення: якщо під час побудови динамічної таблиці стає очевидно, що відстань уже перевищує допустимий поріг помилки, подальші обчислення припиняються.

```
function levenshtein(a: string, b: string, maxErr?: number): number {
  const dp = Array.from({ length: a.length + 1 }, (_, i) =>
    Array.from({ length: b.length + 1 }, (_, j) => (i === 0 ? j : j === 0 ? i : 0)));
  for (let i = 1; i <= a.length; i++) {
    let rowMin = Infinity;
    for (let j = 1; j <= b.length; j++) {
      dp[i][j] = a[i - 1] === b[j - 1]
        ? dp[i - 1][j - 1]
        : 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
      rowMin = Math.min(rowMin, dp[i][j]);
    }
    // Ранній вихід: якщо навіть мінімальне значення рядку > maxErr, подальші немає сенсу
    if (maxErr !== undefined && rowMin > maxErr) return maxErr + 1;
  }
  return dp[a.length][b.length];
}
```

Рисунок 3.4.3 - Алгоритм Левенштейна

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

На основі цього алгоритму реалізовано функцію, яка перевіряє слова пошукового запиту відносно тексту результату із використанням ковзного вікна різної довжини. Це дозволяє знаходити схожі фрагменти навіть тоді, коли довжина слова у запиті та в тексті не збігається повністю. Кількість допустимих помилок визначається відносно довжини аналізованого фрагмента, що забезпечує гнучкий баланс між точністю та стійкістю до помилок.

Результат такої клієнтської обробки обчислюється через useMemo, тобто лише при зміні масиву результатів або пошукового запиту. Завдяки цьому вдається уникнути зайвих перерахунків при кожному повторному рендері сторінки та зберегти прийнятну продуктивність інтерфейсу.

3.4.4 Нормалізація рядків і робота з дефісними словами

Для підвищення точності порівняння текстових значень у клієнтській частині реалізовано механізм нормалізації рядків. Перед виконанням порівняння текст переводиться до нижнього регістру, а дефіси видаляються. Такий підхід дозволяє однаково інтерпретувати варіанти запису на кшталт «вебдодаток» і «веб-додаток».

Нормалізація є важливим етапом, оскільки в реальних даних однакові поняття можуть бути записані в різних формах. Без попереднього приведення до єдиного вигляду навіть незначні розбіжності в написанні могли б погіршувати якість пошуку та підсвічування збігів.

3.4.5 Компонент підсвічування знайдених збігів

Для наочного відображення результатів пошуку в інтерфейсі реалізовано окремий компонент HighlightText, який відповідає за підсвічування знайдених слів у тексті. Його завданням є візуально виділити ті фрагменти теми або опису, які найбільше відповідають введеному користувачем запиту.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Алгоритм роботи компонента передбачає попередню токенизацію тексту на окремі слова за допомогою регулярного виразу. Для дефісних слів додатково формуються злиті токени, що дозволяє коректно працювати як із повною формою, так і з її складовими частинами. Далі для кожного слова запиту визначаються можливі збіги у тексті на основі спільного префікса або нечіткого порівняння через відстань Левенштейна.

```
const HighlightText = ({ text, field: _field, query }: { text: string; matches?: any[]; field: string; query: string }) => {
  if (!text || !query.trim()) return <>{text}</>;

  const queryWords = parseQueryWords(query);
  if (queryWords.length === 0) return <>{text}</>;

  const textLower = text.toLowerCase();

  const wordTokens: { start: number; end: number; lower: string }[] = [];
  const wordRegex = /[a-яієа-з']+/gi;
  let match: RegExpExecArray | null;
  while ((match = wordRegex.exec(textLower)) !== null) {
    wordTokens.push({ start: match.index, end: match.index + match[0].length - 1, lower: match[0] });
  }
}
```

Рисунок 3.4.5 - Компонент підсвічування знайдених збігів

Якщо в межах одного текстового фрагмента знайдено декілька потенційних збігів, пріоритет надається найдовшому токену, щоб уникнути часткового або надмірного підсвічування. Після цього знайдені діапазони сортуються, об'єднуються та відображаються через тег <mark> із візуальним стилем підсвічування. У результаті користувач одразу бачить, чому саме певний запис було показано серед результатів пошуку.

3.4.6 Відображення результатів

Результати пошуку у клієнтському застосунку відображаються у вигляді окремих карток. Кожна картка містить тему дипломного проєкту, статус виконання, інформацію про студента, дані про наукового керівника та, за наявності, короткий

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

опис. Знайдені слова в тексті теми або інших полях підсвічуються, що покращує наочність і допомагає швидше орієнтуватися в результатах.

Для візуального відображення статусу використовуються кольорові бейджі. Наприклад, статус «Захищено» позначається зеленим кольором, а статус «В процесі» — синім. Таке кольорове кодування дає змогу користувачеві швидко оцінити стан проєкту без необхідності детального читання всього запису.

Додатково реалізовано анімацію появи карток за допомогою CSS. Елементи відображаються з ефектом поступового виникнення, а затримка анімації залежить від порядкового номера картки. Це створює каскадний ефект появи результатів, що робить інтерфейс більш плавним і візуально приємним.

3.5 Адміністративна панель

Окремим елементом клієнтської частини системи є адміністративна панель, доступна за маршрутом `/admin`. Її призначення полягає у спрощенні процесу імпорту даних до системи з Excel-файлів, що містять інформацію про розподіл дипломних робіт.

Адміністративний інтерфейс надає можливість завантажувати файл як через механізм `drag-and-drop`, так і через стандартний діалог вибору файлу. Після вибору документа користувач може ініціювати його надсилання на сервер, де відбувається подальша обробка та збереження інформації до бази даних.

Передавання файлу здійснюється через `multipart/form-data` POST-запит до ендпоінту `/upload`. Після завершення імпорту користувач отримує зворотний зв'язок у вигляді повідомлення про результат операції, зокрема кількість успішно оброблених записів або текст помилки у випадку некоректного файлу чи збою під час обробки. Наявність такого адміністративного розділу спрощує наповнення системи початковими даними та дозволяє оперативно оновлювати інформацію без прямого втручання в базу даних.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

3.6 Практична цінність системи

Розроблена система має практичну цінність, оскільки розв'язує реальну задачу, що щороку виникає в діяльності університетських кафедр, а саме облік, систематизацію, пошук та перегляд актуальної інформації про проекти. У традиційному підході така інформація, як правило, зберігається у вигляді Excel-файлів або текстових документів, що ускладнює швидкий доступ до потрібних відомостей, підвищує ризик помилок і потребує значних витрат часу на ручний перегляд матеріалів.

Практична користь системи полягає насамперед у тому, що вона автоматизує роботу з даними та переводить її у структуроване цифрове середовище. Завдяки реалізованому механізму нечіткого пошуку користувач може знаходити релевантні дипломні роботи навіть у випадках, коли пошуковий запит містить помилки або неточності. Це особливо важливо під час пошуку складних технічних термінів, назв методів, алгоритмів або спеціалізованих понять, у написанні яких часто трапляються друкарські похибки.

Ще однією важливою перевагою є ідемпотентний механізм імпорту даних. Повторне завантаження одного й того самого файлу не створює дублікати записів, а лише оновлює вже наявну інформацію. Такий підхід дозволяє безпечно актуалізувати базу даних у разі змін в офіційних таблицях розподілу дипломних робіт та мінімізує ризик накопичення надлишкових або суперечливих даних.

Суттєве значення має також реалізація залежних фільтрів. Завдяки цьому користувачеві пропонуються лише ті значення параметрів, які мають реальні відповідності у вибраній множині даних. Це зменшує кількість зайвих дій під час пошуку, спрощує взаємодію з інтерфейсом і виключає формування некоректних або порожніх комбінацій фільтрів.

Практична цінність системи підсилюється ще й тим, що вона не потребує ручного внесення інформації про студентів, викладачів і теми дипломних робіт.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

Основні дані імпортуються безпосередньо з Excel-файлів, які вже використовуються в межах навчального процесу. Це дозволяє уникнути подвійного введення однакової інформації, зменшує ймовірність людських помилок та скорочує час на початкове наповнення і подальше оновлення системи.

Окремо слід відзначити механізм підсвічування знайдених збігів у результатах пошуку. Він допомагає користувачеві швидко зрозуміти, чому саме певний запис було віднесено до релевантних, і тим самим знижує когнітивне навантаження під час роботи з результатами. Це робить взаємодію з системою більш наочною та зрозумілою.

Додатковою перевагою є простота розгортання системи. Використання SQLite як бази даних та відсутність залежності від зовнішніх сервісів дозволяють запускати застосунок без складної інфраструктурної підготовки. Система може бути розгорнута як на сервері, так і на локальному комп'ютері, що робить її зручною для використання в умовах кафедри або невеликого навчального підрозділу.

Отже, практична цінність розробленої системи полягає в автоматизації обробки даних про дипломні проєкти, скороченні часу на пошук потрібної інформації, підвищенні точності доступу до даних та спрощенні щоденної роботи працівників кафедри.

3.7 Галузь застосування

Основною галуззю застосування розробленої системи є університетська освіта. Насамперед вона може використовуватися на кафедрах технічних, природничих та інших спеціальностей для ведення обліку дипломних, бакалаврських, магістерських і курсових робіт. Система дає змогу централізовано зберігати інформацію про теми, студентів і наукових керівників, а також швидко знаходити потрібні записи за різними критеріями.

Окрім кафедр, система може бути корисною для деканатів, де виникає потреба в узагальненому перегляді та аналізі тематики студентських робіт на рівні

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

факультету. Вона також може застосовуватися в методичних підрозділах для виявлення повторюваних або подібних тем між різними роками навчання, спеціальностями чи кафедрами.

Суміжною сферою використання є інші освітні установи, зокрема коледжі та технікуми, де існує аналогічна потреба в обліку курсових і кваліфікаційних робіт студентів. Подібний підхід може бути адаптований і для аспірантури або докторантури, де система може використовуватися для моніторингу тем дисертаційних досліджень та наукового керівництва.

Крім освітнього середовища, система має потенціал для застосування у корпоративному секторі. Зокрема, її концепція може бути використана у відділах досліджень і розробки для централізованого обліку внутрішніх дослідницьких проєктів, а також у HR-підрозділах підприємств для формування й ведення бази тем для стажерів, практикантів або дипломників, які виконують роботи на базі компанії.

Таким чином, система має не лише вузьке кафедральне призначення, а й ширший потенціал застосування в освітніх і прикладних організаційних середовищах, де є потреба у структурованому зберіганні та пошуку тематичних проєктних даних.

3.8 Масштабованість та напрямки розвитку

Поточна реалізація системи орієнтована на використання в межах окремої кафедри, тобто на роботу з обсягом даних у межах кількох тисяч записів. Для такого сценарію обрана архітектура є достатньо ефективною, забезпечує прийнятну швидкодію та не потребує складної серверної інфраструктури. Водночас структура системи спроектована таким чином, щоб у подальшому забезпечити її масштабування та розширення функціональних можливостей.

У разі збільшення обсягу даних до рівня факультету або всього університету доцільним напрямком розвитку є перехід від SQLite до більш потужної серверної системи управління базами даних, наприклад PostgreSQL або MySQL. Оскільки

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

доступ до даних у системі реалізовано через Prisma ORM, така зміна може бути виконана без суттєвого перепроєктування прикладної логіки, переважно на рівні конфігурації підключення та міграцій схеми бази даних.

Ще одним важливим напрямком удосконалення є оптимізація пошукового механізму для роботи з великими обсягами інформації. Зі зростанням кількості записів доцільно перенести частину функцій повнотекстового пошуку на рівень самої СУБД, використовуючи вбудовані засоби індексації. Це дозволить підвищити швидкодію системи, скоротити навантаження на клієнтську та серверну частини та забезпечити більш ефективну обробку складних запитів.

Крім того, при подальшому масштабуванні доцільним є впровадження пагінації результатів пошуку, кешування часто повторюваних запитів, а також оптимізації запитів до бази даних. Такі зміни дозволять зберігати стабільну продуктивність системи навіть при значному збільшенні кількості користувачів і записів.

Одним із перспективних напрямків розвитку системи є її інтеграція в офіційний вебсайт університету як централізованого механізму роботи з інформацією про дипломні проекти. У такому випадку розроблений застосунок може поступово замінити традиційне використання окремих Excel-файлів, Word-документів та інших розрізнених джерел збереження даних. Користувачі зможуть і надалі виконувати ті самі основні дії, що й раніше, зокрема переглядати, шукати, фільтрувати та оновлювати інформацію про дипломні роботи, однак ці процеси будуть реалізовані в межах єдиного вебсередовища університету. Такий підхід забезпечить централізацію доступу до даних, спростить їх актуалізацію, зменшить дублювання інформації та підвищить загальну зручність користування системою для студентів, викладачів і працівників кафедри.

Перспективним є також функціональне розширення системи. Зокрема, у майбутньому можуть бути додані механізми автентифікації та авторизації користувачів, розмежування прав доступу, журналювання змін, формування статистичних звітів, експорту даних, а також підтримка кількох кафедр або

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

факультетів у межах єдиної системи. Це дозволить перетворити розроблений застосунок із локального кафедрального інструмента на повноцінне університетське інформаційне рішення.

Отже, система має достатній запас гнучкості для подальшого масштабування та розвитку. Поточна архітектура є придатною для практичного використання на кафедральному рівні, а закладені технічні рішення створюють основу для її подальшої інтеграції, розширення та впровадження в межах більшого освітнього середовища.

						Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		

